Week 2: Chunking

- **Chunking** is the process of breaking down large pieces of text into smaller segments.
- The main reason for chunking is to ensure we're embedding a piece of content with as little noise as possible that is still semantically relevant.
- If the chunk of text makes sense without the surrounding context to a human, it will make sense to the language model as well.
- Finding the optimal chunk size for the documents in the corpus is crucial to ensuring that the search results are accurate and relevant.
- Conversational agents - the embedded chunks to build the **context** for the conversational agent based on a knowledge base that **grounds** the agent in trusted information
- **Embedding short and long content –**
    - When we embed our content, we can anticipate distinct behaviors depending on whether the content is short (like sentences) or long (like paragraphs or entire documents).
    - When a **sentence** is embedded, the resulting vector focuses on the sentence's specific meaning.
        - the embedding may miss out on broader contextual information found in a paragraph or document.
    - When a **full paragraph or document** is embedded, the embedding process considers both the overall context and the relationships between the sentences and phrases within the text.
        - a more comprehensive vector representation that captures the broader meaning and themes of the text.
        - Larger input text sizes, on the other hand, may introduce noise or dilute the significance of individual sentences or phrases, making finding precise matches when querying the index more difficult.
    - The length of the query also influences how the embeddings relate to one another.
    - A shorter query will concentrate on specifics and may be better suited for matching against sentence-level embeddings.
    - A longer query may be more in tune with embeddings at the paragraph or document level because it is likely looking for broader context or themes.
    - The index may also be non-homogeneous and contain embeddings for chunks of varying sizes.
- Chunking Models:
    - Fixed-size chunking

```
text = "..." # your text
from langchain.text_splitter import CharacterTextSplitter
text_splitter = CharacterTextSplitter(
    separator = "\n\n",
    chunk_size = 256,
    chunk_overlap  = 20
)
docs = text_splitter.create_documents([text])
```

- “Content-aware” Chunking
- Sentence Splitting
  - NLTK
  - Naïve splitting
  - spaCy
- Recursive Chunking
- Specialised chunking
- Semantic Chunking